The 030 Scaling Study

Loss against data, size and compute on the per-entry trigenic build: the grid queued on Delta, what it records, how it is read, and what each fit can and cannot support

Michael Volk

October 12, 2026

Abstract

Forty one-GPU jobs on Delta train the per-entry cell graph transformer of experiment 030 across five sizes (79k to 3.6M non-embedding parameters) and four nested training subsets of the strict closure pool (21k to 1.05M records, drawn by query pair), with the merged-label 025 twin trained on the same subsets at the largest size. Every run logs the three scaling quantities as it trains: N from the model's own parameter tally, D as distinct records per epoch and records seen, and C as FLOPs measured on five real training steps rather than taken from $6ND$. The readout scores every epoch on the pinned 37,705 validation triples in label units, pooled and per screen, takes two reads per point (the validation-selected epoch and an interpolation at one shared records-seen budget), and fits the pure and floored power laws of the scaling-laws explainer on both reads, with the largest point held out and a bootstrap over runs for every exponent, beside the replicate floor of each screen. Three defects that would have stopped every figure were found and repaired before any job started: an override Hydra refused, compute rows a W&B history query could not see, and a per-screen loss key the trainer never logged. What the queued runs support, in order of confidence: the paired gap at the two smallest subsets, loss against unique data at a fixed training budget, and a fixed-epoch slice of loss against size. The unconverged full-data points and a loss range less than three times the replicate floor are the limits on every exponent, and the cheapest additions that move them are listed last.

Section status: ✓ ready ■ author-reviewed, keep stable ✗ not done, agents may edit freely

Contents

1 The question and the vocabulary ✗ **2**
2 The grid ✗ **2**
3 The readout ✗ **4**
4 What each fit can support ✗ **6**
5 The planned figures ✗ **7**
6 What to run next, by what it buys ✗ **8**

1 The question and the vocabulary ✗

Measured. The grid is queued on Delta (calibration 22828396 and 22828397, the 030 grid 22812513 to 22812546, the 025 pair 22812547 to 22812556, all pending on 2026-10-12). The parameter counts, subset sizes, budgets and replicate floors in the tables are read from the grid's own files. The three defects of Section 4 were each reproduced before being fixed.
Not measured. Every exponent, every floor-against-fit comparison and the paired gap. No run of the grid has completed. The curves in the planned figures are sketches of what each panel will hold.

A **scaling law** is a fitted relation between a model's held-out loss L and what a training run spends: trainable parameters N, distinct training examples D, and compute C. The two published forms, Kaplan's single-axis power law $L = (N_c/N)^{\alpha_N}$ and Hoffmann's joint form with a floor, $L(N, D) = E + AN^{-\alpha} + BD^{-\beta}$, and the recipe for fitting them (least squares in log space from a grid of starts, a Huber residual, the largest run held out, a bootstrap over runs) are set out in the scaling-laws explainer (`notes-tex/modeling/scaling-laws`, with the forms drawn and the fitting functions exercised on synthetic runs in `experiments/041-scaling-laws/scripts/scaling_law_forms.py`). The three works behind those forms are in the mirror under `kaplanScalingLawsNeural2020`, `hoffmannTrainingComputeOptimalLarge2022` and `besirogluChinchillaScalingReplication2024`. This study is the first set of runs on a torchcell model built to be fit against them.

The question the grid answers is the one that decides where the next unit of effort goes on the trigenic build: does the loss still fall with more distinct data at the current size, does it still fall with more parameters at the current data, and how far is either curve from the floor the replicate noise of the screens sets. A fourth question is specific to the 030 build: what keeping every stored measurement of a genotype, under a token that names its source screen, buys over one merged label per genotype, read as the gap between the 030 curve and its 025 twin at every data size.

Vocabulary. Terms with a technical sense here, in alphabetical order. The loss, the data unit and the two reads are the ones a reader has to hold apart.

term	meaning
calibration job	a 3 h job of 400 training batches at one size on the full pool, run before the grid, whose logged records seen, GPU hours and FLOPs per step give the pace the epoch budgets assume; two of them, at 180x8 and 45x2, so that equal paces expose a loader-bound run.
closure double	a double-deletion record whose gene pair lies inside a training triple's gene set; the strict pool S3Q keeps every closure double except the 88 whose pair is itself a held-out query pair.
D, records and entry rows	distinct training examples. On 025 the unit is the record (one merged label per genotype); on 030 the per-entry loss averages over entry rows, every stored measurement of a genotype, 1.6 to 2.2 rows per record across the subsets. Both units are reported; repeated epochs are not more D.
dataset token	the 030 arm's one-hot over the build's 15 source datasets, projected to 8 dimensions and concatenated to the readout, so every entry row trains under the name of its screen and every validation triple is scored under its own screen's token.
E, replicate floor	the expected squared error of the best possible predictor of a single noisy measurement, $(1-r)\,\mathrm{var}(\tau)$ for replicate reliability r, per screen in label units (Table 4); the lower bound on the E of a floored fit.
held-out test	the largest point of an axis is left out, the form is fit on the rest, and its prediction at that point is compared with the observation; the test of whether an exponent extrapolates.
label units	the gene-interaction score τ after the inverse of the training normalization, so a loss is comparable across sizes, subsets and builds; the normalized loss is not, because its constants are fitted per training set.
matched read	the loss of a point at one shared records-seen budget B, interpolated on its per-epoch validation curve; B is the smallest total any point of the grid reached, so every point has the read.
N, non-embedding	trainable parameters outside the gene embedding table and the composite-embedding preprocessor, the count the loss responds to; 79k to 3.6M across the ladder (Table 1).
query pair	the two query genes of a trigenic screen's triple; the unit that leaks between training and evaluation, so the evaluation triples are held out with their query pair and the subsets are drawn by it.
screen	one of the two trigenic sources, Kuzmin 2018 and Kuzmin 2020, whose validation triples never overlap and whose means differ by about one standard deviation, which is why curves are fitted per screen.
selected read	the loss of a point at the epoch of lowest pooled validation MSE, the validation-selected checkpoint; the converged read for subsets that converge inside their budget.

2 The grid ✗

Every 030 run is the composite closure arm on the query-pair-disjoint split, `cgt_030_s3_q_tok_embfit_004`: the per-entry trainer with the dataset token, the strict pool S3Q, every stored entry a training row, scored on the pinned

37,705 validation triples. What varies is the size and the training subset (Figure 1). Every 025 run is the same arm's merged-label twin, `cgt_s3_q_kl_embfit_037`, on the same subsets at the current size. Batch is 256 records on one A40, the learning rate 2.5e-4 constant, the normalization constants those of the full pool on every subset (`transforms.fit_stats_strict=false`), so the optimization target does not move with D on 030. The 025 trainer has no such switch and refits its constants on each subset's training records; the losses are compared in label units, where that difference does not enter, but the training objectives differ on that side of the pair.

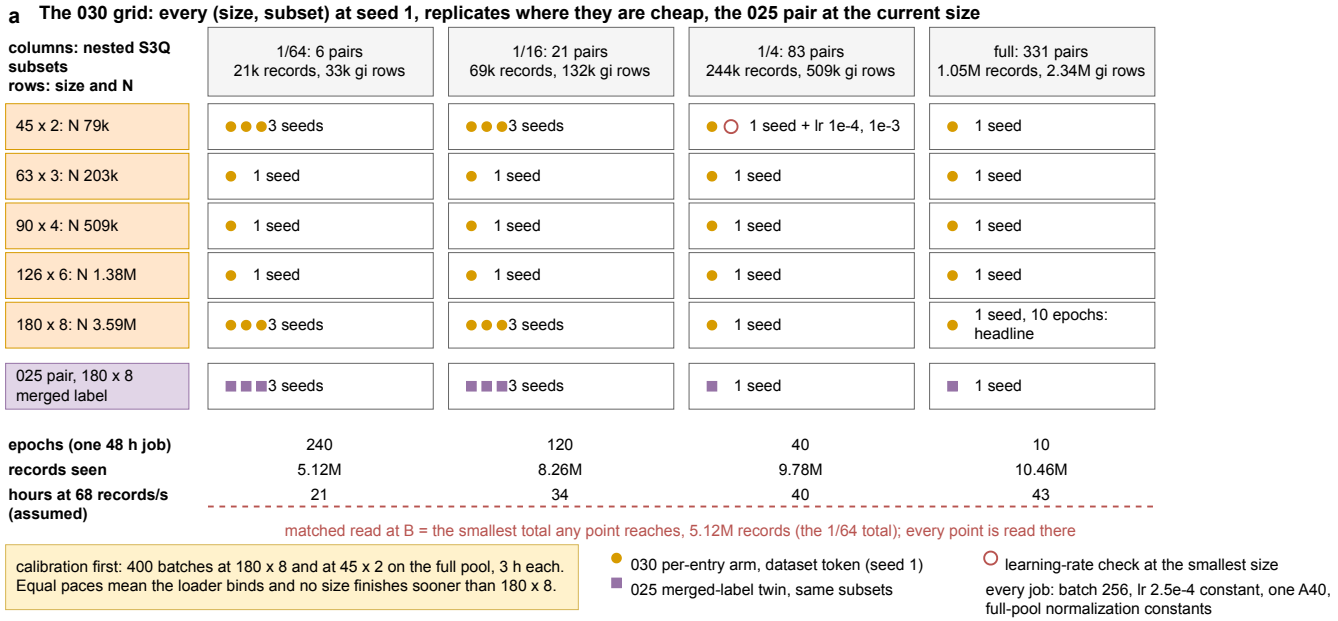


Figure 1: The grid as queued. Columns are the nested subsets of S3Q, drawn by query pair with seed 42 so the 1/64 pairs lie inside the 1/16 pairs inside the 1/4 pairs; rows are the five sizes with their non-embedding N. A filled mark is a queued run; three marks are seeds 1 to 3, placed where runs are cheapest (the two smallest subsets) at the two end sizes; the red ring is the learning-rate check at the smallest size on 1/4. The purple row is the 025 twin at 180x8. The strip below gives each column's epoch budget, the records seen under it, and the wall time at the assumed pace; the dashed line is the shared budget of the matched read. Planned figure drawn from the grid's own tables; no run has reported.

SRC: `notes/assets/drawio/030-scaling-study.gen.py`

The size ladder. Width and depth scale together in the current configuration's ratio at nine attention heads (Table 1): 45x2, 63x3, 90x4, 126x6 and 180x8, the last being the size every 030 and 025 arm has trained at. N is read from the model's own tally at each size, the count the compute callback logs, and the 180x8 count equals the one W&B holds for the instrumented IGB run. The ladder spans 1.66 decades, under the two decades the explainer's recipe asks for, and the composite preprocessor that projects the sequence embeddings to the hidden width is excluded from N as the embedding is.

Table 1: The N ladder. Width and depth scale together at 9 attention heads; N is the non-embedding count the compute callback logs, read from the model's own tally at each size, with the gene table and the composite preprocessor excluded. The 180x8 count equals the one W&B holds for the instrumented IGB run (53qqijnu, 3,587,350), asserted by the generating script.

width × depth	head dim	N (non-embedding)	$N/N_{180\times8}$	$\log_{10} N$
45 × 2	5	79,375	0.022	4.90
63 × 3	7	202,612	0.056	5.31
90 × 4	10	509,260	0.142	5.71
126 × 6	14	1,378,444	0.384	6.14
180 × 8	20	3,587,350	1.000	6.55

The data ladder. Subsets are drawn by the unit that leaks, the query pair, so a smaller subset removes whole query screens with every triple they reach, and the closure doubles follow their triples (Table 2). Singles are in every subset, so the single-mutant fitness supervision does not vary with D, and the pinned validation and test triples are in every pool unchanged. The same shuffle on the same sorted pair list gives the same kept pairs on the 025 build, pair for pair, so a 030 subset and its 025 twin hold the same genotypes and differ only in what is kept per genotype. D is counted in both units. The ratio between them drifts from 1.56 rows per record at 1/64 to 2.24 at the full pool, because the Costanzo and Kuzmin double-mutant screens that supply most of the extra rows cover the recurring pairs more densely; the 030 curve is therefore fitted in both units and the two exponents reported side by side.

Table 2: The D ladder and its budgets. Subsets are nested by query pair (seed 42, the same kept pairs on the 025 build); singles are in every subset and the pinned validation and test triples are untouched. Records are the 025 unit of D; gene_interaction entry rows are the 030 unit, what the per-entry loss averages over, and rows per record is the ratio between them. Epochs are the submit script's budget, records seen is records × epochs, and hours is that count at the ASSUMED pace of 68 records/s on one A40 at 180x8 plus stage-in, pending the calibration jobs.

subset	pairs	triples	doubles	records	gi rows	rows/rec	epochs	records seen	h at pace
1/64	6	5,211	10,428	21,333	33,348	1.56	240	5.12M	20.9
1/16	21	21,045	42,088	68,827	131,613	1.91	120	8.26M	33.8
1/4	83	79,677	159,031	244,402	509,458	2.08	40	9.78M	40.0
full	331	301,236	739,148	1,046,078	2,340,771	2.24	10	10.46M	42.8

Budgets. Every point is one 48 h job, because a dependent job accrues no queue priority on Delta and a chain pays a fresh queue wait per link. The epoch budget is keyed by subset: 240, 120, 40 and 10 epochs for 1/64 through the full pool, which is what 48 h holds at the 180x8 pace for the two largest subsets. That pace, 68 records per second on one A40, is read from the 025 four-GPU runs and is an assumption until the calibration jobs report; the first version of the submit script assumed four times that and would have timed out every full-data job before its tenth epoch. The budget table is keyed by subset only, so a small size on the full pool gets the same ten epochs as 180x8 whether or not it could have trained longer, which Section 4 returns to. The 180x8 full-data point at ten epochs is the headline point of both the D and N axes and is not converged: the same arm on IGB was at validation Pearson 0.384 over epochs 13 to 22 and still rising, and the random-split seed took about 150 epochs to turn.

What is recorded per run. The compute callback (`torchcell.trainers.compute_accounting`) writes N at fit start, and at every epoch end the records per epoch, the cumulative records seen, the measured FLOPs per step and their cumulative sum, and the GPU hours, together with the epoch index. FLOPs are counted by `torch.utils.flop_counter.FlopCounterMode` around five real training steps (batch indices 1 to 5, forward and backward), and the mean, the minimum and the maximum are logged, so a per-entry batch whose row count varies shows its spread rather than one draw; $6ND$ is a rule for token transformers and does not describe a gene-token model with graph penalties. The count covers the matmul family only and is a lower bound, comparable across sizes of one architecture. The callback's state is saved with the checkpoint, so a chain of jobs continues its totals.

Table 3: The job list, 32 jobs of the 030 grid and 8 of the 025 pair, one GPU each on the Delta A40 partition, 48 h apiece except the two 3 h calibration jobs (400 training batches at 180x8 and 45x2, which report the pace and the measured FLOPs per step). Every 030 job is the composite closure arm on the query-pair-disjoint split (cgt_030_s3_q_tok_embfit_004) at batch 256; every 025 job is its merged-label twin (cgt_s3_q_kl_embfit_037) at 180x8.

sweep	what varies	jobs	sizes	subsets	seeds
030_sclw1	calibration	2	45x2, 180x8	full	1
030_sclw1	size × subset	20	45x2, 63x3, 90x4, 126x6, 180x8	1/64, 1/16, 1/4, full	1
030_sclw1	seed replicates	8	45x2, 180x8	1/64, 1/16	2, 3
030_sclw1	learning-rate check (1e-4, 1e-3)	2	45x2	1/4	1
025_sclw1	subset	4	180x8	1/64, 1/16, 1/4, full	1
025_sclw1	seed replicates	4	180x8	1/64, 1/16	2, 3

3 The readout ✗

The readout (`scaling_readout_030.py`) finds every run of a sweep by its config tag, stitches chained continuations by the checkpoint each link resumed from, and keeps one row per epoch of every point with the validation loss in label units, pooled and per screen, the Pearson beside it, and the compute keys (Figure 2). A point is one (size, subset, seed, learning rate).

SRC: `notes/assets/drawio/030-scaling-study.gen.py`

The loss that is fitted. The fit target is the mean squared error on the gene-interaction score after the inverse transform, `val/gene_interaction/MSE`, and per screen the per-token keys `val/token/[screen]/gene_interaction/MSE`. Not the normalized loss, whose constants differ per arm, and not Pearson, which plateaus and carries a floor of its own: the two screens' validation triples differ in mean by about one standard deviation (Kuzmin 2018 mean −0.046, sd 0.049, $n = 10{,}067$; Kuzmin 2020 mean +0.004, sd 0.062, $n = 28{,}935$, measured on the Q validation entries), so predicting only the per-screen mean scores a pooled Pearson of 0.34 and every pooled Pearson carries that component. Squared error does not, but the two screens still have different replicate floors, so the curves are fitted per screen and the pooled curve is read as their mixture. The per-token MSE is computed over validation entry rows (39,002 rows of the 37,705 triples),

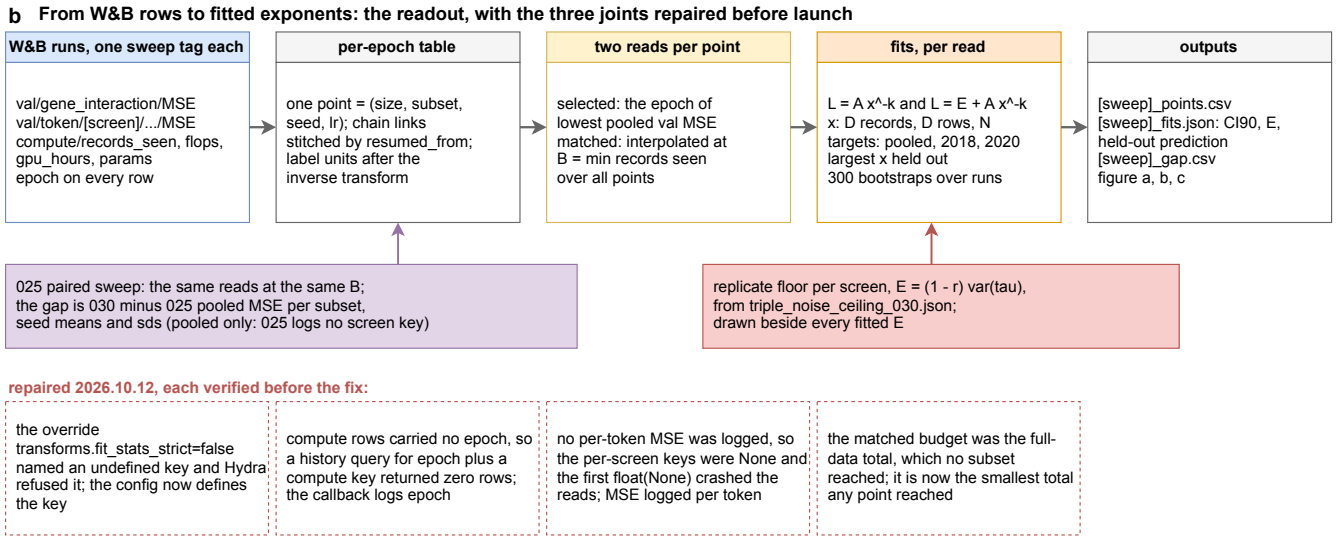


Figure 2: The readout from W&B rows to fitted exponents. The five boxes are the stages; the purple box is the 025 paired sweep, which enters the same table and leaves as the gap; the red box is the replicate floor, computed before any fit and drawn beside every fitted E. The dashed boxes are the four joints repaired on 2026-10-12, each of which had been reproduced on a real run or a real compose before it was changed (Section 4).

the pooled MSE over the policy-reduced one value per triple; the populations differ by the triples measured more than once in one screen.

Two reads per point. The selected read is the epoch of lowest pooled validation MSE. For the subsets that converge inside their budget (240 epochs of 1/64, 120 of 1/16) it is the converged loss; for the full pool at ten epochs it is the last epoch or close to it. The matched read interpolates each point's epoch curve at one records-seen budget B shared by every point. B defaults to the smallest total any point reached, which the budgets make the 1/64 total of 5.12M records: the full pool at ten epochs sees 10.46M, 1/4 at 40 epochs 9.78M, 1/16 at 120 epochs 8.26M, so a budget set at the full-data total would leave every subset without the read, which is how the first version of the readout was written. At $B = 5.12$M the full point is read near its fifth epoch and 1/4 near its twenty-first. The matched read is loss against unique data at a fixed training budget, the data-constrained framing in which repeated epochs of a small subset are set against single passes over a large one.

Fits. On each read and for each target (pooled, 2018, 2020), two forms on each axis: the pure power law $L = Ax^{-k}$ and the floored form $L = E + Ax^{-k}$, where x is D in records, D in entry rows (030 only) at the 180x8 column, and N at the full-pool column. The fit is least squares in log space from a grid of starting exponents, with a Huber residual at $\delta = 10^{-3}$ as Hoffmann et al. fit, and E bounded between zero and the smallest observed loss. For every fit the largest x is held out, the form is refit on the rest, and its prediction at the held-out point is reported beside the observation; the floored form has three parameters, so this test needs four distinct x in sample, and on three it is an exact fit that predicts nothing. Three hundred bootstrap resamples over runs give the 5 to 95 percent interval of every exponent. The interval, not the point, is what an extrapolation inherits.

The floor before any fit. Under a squared loss the best predictor of a single noisy measurement has expected error equal to the replicate variance, so the released and re-measured records of the screens bound E from below in the loss's own units before a fit exists (Table 4). The 2020 reliability is measured on this build's 11,013 same-strain re-measured triples; 2018 has 61 such pairs, so it is bounded by the published Dango replicate correlation below and by the 483 cross-year pairs above. On 2020 the floor is 56% of the variance of the score: no predictor removes more than 44% of the loss of predicting the screen mean. A fitted E below the floor is a wrong fit; one far above it says the model family, not the data, is the limit.

The paired gap. The 025 sweep passes through the same table and the same two reads at the same B. The gap at each subset and read is the 030 pooled MSE minus the 025 pooled MSE, each a mean over seeds with the seed standard deviation where there are three. It is pooled only: the 025 trainer logs no per-screen key, and its validation triples are the same genotypes so a per-screen 025 loss would need its predictions dumped and scored offline.

Table 4: The replicate floor per screen in label units. Under a squared loss the best predictor of a single noisy measurement has expected error $E = (1 - r)\,\mathrm{var}(\tau)$ where r is the replicate reliability, the correlation of two independent measurements of the same triple. The variance column is the loss of predicting the screen mean, the top of the range a curve can move through, so the floor's share of it is the fraction of the loss no model can remove. The 2020 reliability is measured on this build's same-strain re-measured pairs; 2018 has 61 such pairs, so it is bounded by the published Dango replicate correlation below and the cross-year 2018 vs 2020 pairs above.

screen	reliability from	r	$\mathrm{sd}(\tau)$	$\mathrm{var}(\tau)$	E	E/var
Kuzmin 2018	Dango 0.59	0.59	0.0535	2.86e-3	1.17e-3	0.41
Kuzmin 2018	cross-year 2018 vs 2020 pairs	0.89	0.0535	2.86e-3	0.31e-3	0.11
Kuzmin 2020	same-strain re-measured pairs	0.44	0.0633	4.01e-3	2.25e-3	0.56

4 What each fit can support ✗

Three defects would have stopped every figure, and each was reproduced before it was fixed. The submit script passed `transforms.fit_stats_strict=false` without the `+` Hydra requires for a key no config defines; composing the exact launch overrides locally reproduced the refusal, and the first calibration job (22812512) printed `Could not override 'transforms.fit_stats_strict'` after thirty minutes of imports from Lustre. The config now defines the key, and since configs and code are read from the Delta checkout when a job starts, a pull repaired every pending job in place. The compute callback logged its keys without an epoch field, and a W&B history query for `epoch` together with `compute/records_seen` returned zero rows on the one instrumented run (53qqijnu) while the key alone returned 28; the callback now logs the epoch on every row and the readout falls back to the epochs-completed counter for older runs. The trainer logged a per-token Pearson but no per-token MSE, so the per-screen keys the readout fits did not exist, and the first missing key crashed the reads on `float(None)`; the per-token collection now carries MSE in label units beside Pearson. A fourth, the unreachable matched budget, is above. None of these touched the model or the training.

With those repaired, the three fits divide by how much the queued runs support.

The paired gap: supported at 1/64 and 1/16. Both builds have three seeds at the two smallest subsets, both converge there, and the reads are at the same B. The gap is interpretable at those two points, with a seed spread on each side. At 1/4 it is one seed each. At the full pool it is the difference of two unconverged ten-epoch runs, a gap in learning speed at ten epochs rather than in attainable loss, and it is drawn hollow. Two confounds belong in its caption: the 025 normalizer refits on each subset while 030 pins the full-pool constants, and the 030 arm differs from 025 by more than the per-entry rows, since it also carries the dataset token and the Kuzmin query-strain double-fitness entries the 025 build never had. The gap is "per-entry rows plus token plus the added measurement types"; separating the token is one more 180x8 column.

$L(D)$ at 180x8: supported on the matched read, in a narrower claim. Four points at 180x8, with three seeds at the two smallest. On the selected read the small subsets are converged and the full pool is not, so the curve flattens at its top and β is biased low with the slack absorbed into E; the minimum over 240 noisy epochs against a minimum over ten biases the same way. On the matched read every point is a loss at $B = 5.12$M records seen, and the fit is the claim it can be: at a fixed budget, what more distinct data buys. The deeper limit is identifiability. The pooled replicate floor works out near 1.7e-3 to 1.9e-3 in label units; the best pooled validation MSE this arm has reached is 3.3e-3 (53qqijnu, epoch 40), and predicting the per-screen mean scores about 3.9e-3. Loss minus floor therefore spans less than a factor of 2.5 across the whole curve, and the floored form has one degree of freedom on four points. The power form's held-out test is the only one that means anything on this axis, and the fitted E is a bound to report with its interval rather than a value.

$L(N)$ at the full pool: a fixed-epoch slice. Five sizes, every one at ten epochs, because the budget table is keyed by subset. If the GPU is the bottleneck, the small sizes finish ten epochs in a fraction of 48 h and could have run to convergence; if the loader is, as it was on IGB's GPFS, every size costs 4.3 h an epoch and ten epochs is what any of them gets. The two calibration jobs tell which. Either way the slice measures loss after 10.5M records seen as a function of N, which conflates how fast a size learns per epoch with how much it can learn, and the learning rate is checked at the smallest size only while every size runs at a batch four times smaller than the one the arm was tuned at. The exponent from this slice is descriptive until the small sizes have been trained to convergence and the learning rate has been checked at the top of the ladder.

The compute axis: not in this grid. IsoFLOP curves need several budgets with several sizes at each, with records seen set by the budget and the per-size cost per record from the calibration jobs. The grid logs everything such a sweep would read, and the fixed-epoch column is one budget line of it, but a frontier needs three.

5 The planned figures ✗

Two figures carry the result: the three-panel results figure the readout writes from the runs (Figure 3 is its sketch), and the reading guide that says what each outcome means for the next unit of effort (Figure 4). Both are drawn before any run has reported, so that the figure is specified before the data exist and the readout writes to it rather than the other way round.

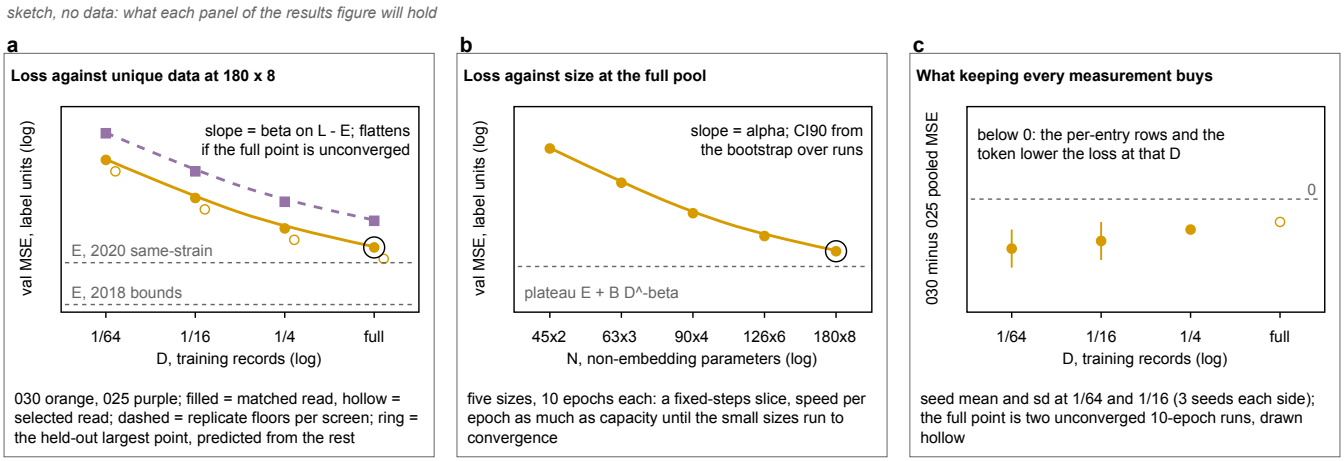


Figure 3: The results figure, sketched. **a**, validation MSE in label units against D at 180x8, 030 in orange and the 025 twin in purple, the matched read filled and the selected read hollow, the replicate floor of each screen dashed, the full point ringed as the held-out point predicted from the rest. The 2018 and 2020 curves are drawn separately in the real figure; one curve stands for them here. **b**, validation MSE against N at the full pool, five sizes at ten epochs each, the plateau the data sets dashed. **c**, the gap, 030 minus 025 pooled MSE per subset, seed mean and spread at 1/64 and 1/16, the full point hollow as two unconverged runs. Every curve is hand-drawn to show what the panel holds; `scaling_readout_030.py` writes the real figure.

SRC: `notes/assets/drawio/030-scaling-study.gen.py`

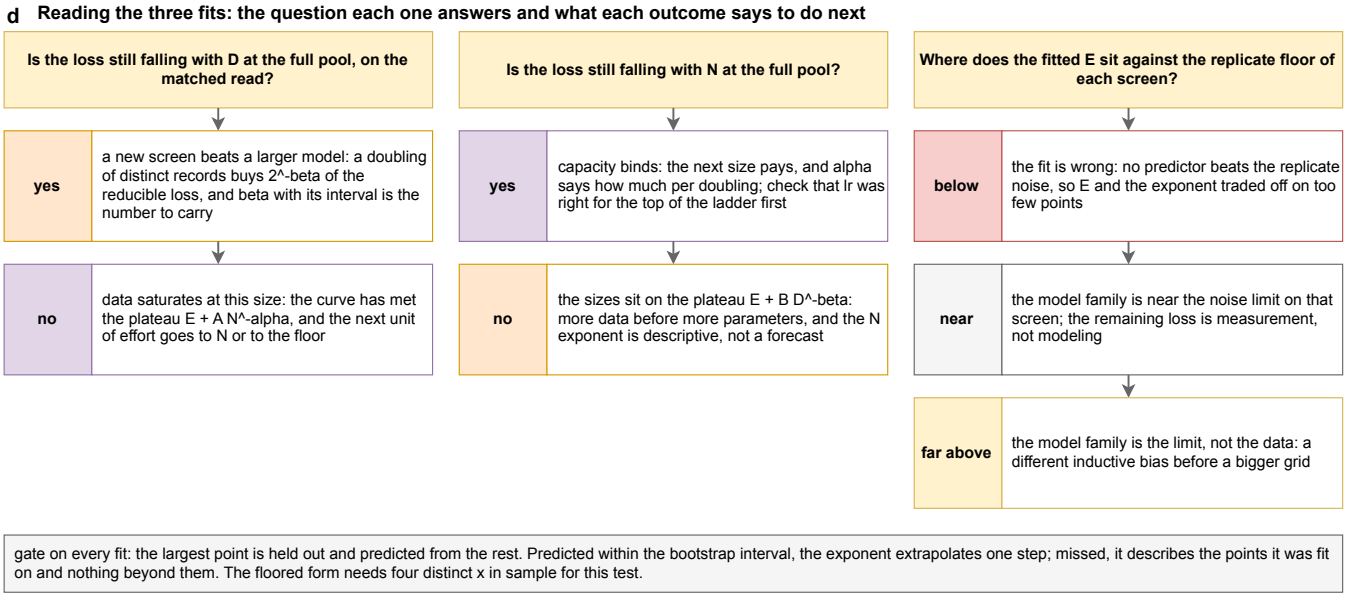


Figure 4: How the three fits are read. Each column is one question with its outcomes and what each outcome says to do next; the colors follow the figure above, orange for an answer that points at data, purple for one that points at size, red for a fit that cannot be right. The gate at the bottom applies to every fit: an exponent whose held-out prediction missed describes the points it was fit on and nothing beyond them.

SRC: `notes/assets/drawio/030-scaling-study.gen.py`

Figure placeholder: the results figure from `scaling_readout_030.py`, three panels at 179 mm, written to `ASSET_IMAGES_DIR/030-solid-growth-multi/scaling_030_sclw1_[timestamp].svg` once the grid has run.

6 What to run next, by what it buys ✗

In order of the limit each one moves, with its cost in 48 h one-GPU jobs. None is queued.

Read the calibration jobs before trusting the budget table. Two 3 h jobs at 180x8 and 45x2 report records per second and the measured FLOPs per step at both ends of the ladder. Equal paces mean the loader binds, every size costs 4.3 h an epoch, and the fixed-epoch slice of $L(N)$ is all 48 h can give; a faster 45x2 means the small sizes can run to convergence on the full pool, and the epoch budget should then scale per size by the ratio of paces. The validation pass is also timed there: at 240 epochs of 1/64 it runs 240 times over 37,705 triples at a validation batch of 32, and if one pass costs more than a quarter of an epoch the small subsets need validation every second epoch to finish inside 48 h, with the readout interpolating on the epochs that have a validation row.

A 1/8 subset at 180x8, both builds. Two jobs. The subsets are nested by one shuffle, so adding a denominator leaves the existing subsets unchanged and adds a fifth point to the D axis between 1/16 and 1/4, which is what the floored form's held-out test needs (four points in sample) and what the gap needs for a third converged point. Eighty epochs of 1/8 is 10.4M records, one 48 h job at the assumed pace.

The learning rate at the top of the ladder. One or two jobs at 180x8 on 1/16 at 1e-4 (and 6e-4). The check at 45x2 on 1/4 says nothing about the end of the ladder where the batch is four times smaller than the arm was tuned at, and an N exponent fit across sizes run at one learning rate is an exponent of the schedule as much as of the capacity.

A converged full-data point. Not cheap: one hundred epochs at 180x8 is nine chained 48 h jobs with a queue wait per link. Until it exists the top of the D axis and the whole N axis are read at ten epochs, and the figures say so. The alternative that costs nothing is to fit the converged curve on 1/64, 1/16, 1/8 and 1/4 only and treat the full point as the held-out prediction target, which the readout already does when the full point is the largest x.

The token ablation. One 180x8 column of the 030 arm with `dataset_token.enabled=false` on the four subsets, four jobs, which separates the token from the per-entry rows in the gap.

The compute axis. Three budgets spanning two decades of FLOPs, four sizes at each, records seen set by the budget and the per-size cost per record from the calibration jobs; twelve jobs, after the first two sweeps have shown that the budget buys anything.